Lesson 10 Teacher's Key: Play It Again

GameMaker - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

GameMaker's actual reusable-function mechanism (a Script resource) - introduced for the first time this course, after eight lessons of event-only code, specifically because this lesson's core idea (one setup routine, called from two places) finally needs it.

Level 1 - Full Worked Solution

- scr_start_game is a genuinely new kind of resource for this course - previous lessons kept all logic inside object Events; this is the first time code lives in a standalone, explicitly reusable Script.
- with (obj_enemy) { instance_destroy(); } is the correct, idiomatic GML way to destroy every instance of a given object type in one line - a new, useful pattern worth calling out explicitly.
- obj_spawner's Create event and obj_gamemanager's Step event (the restart check) both call the exact same start_game() - the two call sites this lesson is built around.

Highlighted lines are new this week; everything else is code you already have.

```
// scr_start_game (Script resource - new):
function start_game() {
    with (obj_enemy) {
        instance_destroy();
    }
    obj_gamemanager.__dnd_score = 0;
    obj_gamemanager.__dnd_lives = 3;
    obj_gamemanager.enemies_remaining = 5;
    obj_gamemanager.has_won = false;
    obj_gamemanager.has_lost = false;
    obj_gamemanager.game_active = true;
    for (var enemy_count = 0; enemy_count < 5; enemy_count++) {
        instance_create_layer(80 + enemy_count * 96, 80, "Instances",
obj_enemy);
    }
}

// obj_spawner's Create event (revised - the spawn loop moved into
start_game()):
start_game();

// obj_gamemanager's Create event:
```

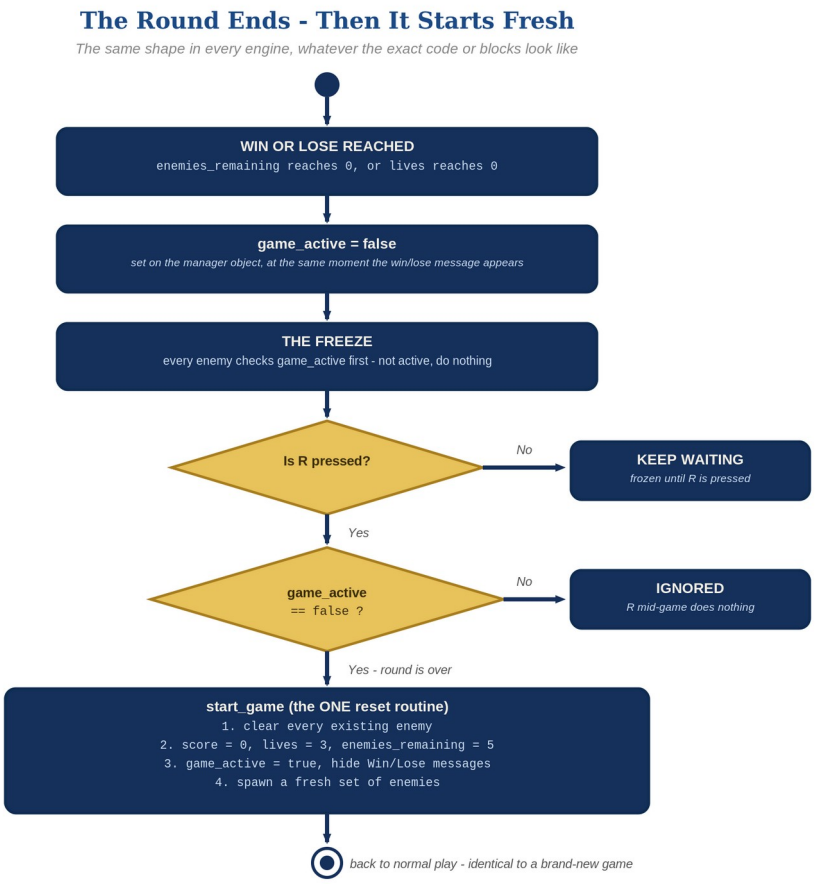
```
enemies_remaining = 5;
has_won = false;
__dnd_lives = 3;
has_lost = false;
__dnd_score = 0;
game_active = true;

// obj_gamemanager's Step event (new):
if (!game_active && keyboard_check_pressed(ord("R"))) {
    start_game();
}

// obj_enemy's Step event (revised):
if (!obj_gamemanager.game_active) exit;
y += enemy_speed;
if (y > room_height) {
    obj_gamemanager.__dnd_lives -= 1;
    if (obj_gamemanager.__dnd_lives <= 0) {
        obj_gamemanager.has_lost = true;
        obj_gamemanager.game_active = false;
        audio_play_sound(sfx_gameover, 1, false);  // your Lesson 8 Level
3 sound
    } else {
        audio_play_sound(sfx_lifelost, 1, false);  // your Lesson 8 Level
3 sound
    }
    instance_destroy();
}

// obj_bullet's Collision event with obj_enemy (revised):
audio_play_sound(sfx_hit, 1, false);
obj_gamemanager.enemies_remaining -= 1;
obj_gamemanager.__dnd_score += 10;
if (obj_gamemanager.enemies_remaining <= 0) {
    obj_gamemanager.has_won = true;
    obj_gamemanager.game_active = false;
    audio_play_sound(sfx_win, 1, false);  // your Lesson 6 Level 3 sound
}
instance_destroy(other);
instance_destroy(self);

// obj_gamemanager's Draw GUI event (revised):
draw_text(20, 20, "Score: " + string(__dnd_score));
draw_text(20, 50, "Lives: " + string(__dnd_lives));
if (has_won) {
    draw_text(400, 50, "You Win! Press R to Restart");
}
if (has_lost) {
    draw_text(400, 90, "Game Over - Press R to Restart");
}
```



start_game also runs once at launch - one setup routine, two doorways in.
Nothing about the setup logic was rewritten for restarting.

VIDEO GAME CREATION

When the round ends, everything freezes - and R, pressed only while the round is over, runs the one reset routine. The same shape in every engine.



scr_start_game, obj_gamemanager's Step event, and obj_enemy's Step event - the block-for-block equivalent of the Level 1 code. (the Visual path)

Why This Structure

- Introducing a Script resource here (rather than earlier) is deliberate: this is the first lesson where the same setup logic genuinely needs to run from two different triggers, which is exactly when a reusable function starts paying for itself.
- keyboard_check_pressed() (edge-triggered, fires once per press) is used rather than keyboard_check() (fires every frame held) for the same reason TIC-80 uses keyp() instead of key() this week - worth drawing this cross-engine parallel for students in both.

Level 2 / Level 3 Talking Points

- Level 3 bug hunt: kills reduce enemies_remaining but escapes only reduce lives, so a round with some kills and some escapes (for example 3 and 2) ends with no enemies

left, no win and no loss, and R does nothing because game_active is still true. This started in Lesson 6 (win = all killed) and Lesson 8 (lose = lives gone). A good fix, and the one shown here, is that an escape also resolves an enemy and the win check requires lives above 0. Accept any design that guarantees every round ends in exactly one outcome (for example a third 'Round Over' state, or a check that no enemies remain and none are in flight). Test it with 3 kills and 2 escapes, then 4 and 1, then 0 and 5.

- Ask students who've been in GameMaker since Lesson 2: why didn't we need a Script resource until now? (Every previous lesson's logic only needed to run from one place - this is the first time the same setup genuinely needs two entry points.)
- For advanced students: point out that Scripts can also take parameters and return values, previewing more advanced GML use beyond what this lesson needs.

Level 2 - complete code:

```
// Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
// Checks: after a win and after a loss, R restores 0/3/5 with no leftover
enemies; R during play does nothing.
```

Level 3 - code (additions highlighted):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
// Level 3 visual: obj_gamemanager Draw GUI event.
if (has_won) {
    draw_set_color(c_yellow);
    draw_set_halign(fa_center);
    draw_text(400, 50, "You Win! Press R to Restart");
    draw_set_halign(fa_left);
    draw_set_color(c_white);
}

// BUG FIX (one good answer): an escape also resolves an enemy.
obj_enemy's Step event, escape branch:
if (y > room_height) {
    obj_gamemanager.__dnd_lives -= 1;
    obj_gamemanager.enemies_remaining -= 1;
    if (obj_gamemanager.__dnd_lives <= 0) {
        obj_gamemanager.has_lost = true;
        obj_gamemanager.game_active = false;
        audio_play_sound(sfx_gameover, 1, false);  // your Lesson 8 Level
3 sound
    } else if (obj_gamemanager.enemies_remaining <= 0) {
        obj_gamemanager.has_won = true;
        obj_gamemanager.game_active = false;
        audio_play_sound(sfx_win, 1, false);  // your Lesson 6 Level 3
sound
    } else {
        audio_play_sound(sfx_lifelost, 1, false);  // your Lesson 8 Level
3 sound
    }
    instance_destroy();
}
// GML Visual: add a Set Variable block (Target obj_gamemanager,
enemies_remaining, Value -1, Relative checked) after the Set Lives block,
and an Else branch holding an If Variable (enemies_remaining <= 0) that
sets has_won and game_active.
```

Common Student Mistakes

- Game-state drift: the student added start-up code or a game-over effect beyond the handout (extra green-flag scripts, an animation, a color change, a hidden object) and did not reset it on restart. Symptom: a restart that looks almost right. Walk through the Handout's "Audit Your Game State" list with them.
- Writing start_game()'s logic directly in obj_spawner's Create event instead of in the new Script resource, then trying to call it from Step anyway (Create-event code isn't callable from elsewhere the way a Script is).
- Placing "with (obj_enemy) instance_destroy();" AFTER the spawn loop instead of before it, destroying the freshly-spawned enemies by mistake.
- Forgetting the obj_gamemanager. prefix when obj_enemy's Step event checks game_active, causing a variable-scope error (game_active lives on obj_gamemanager, not obj_enemy).